

Week 09 Day 1 Assignment:

Prompt:

Instruction:

Help me to design architecture for identifying regression impacted areas and regression test cases to be added to the regression suite based on the PRs merged during the sprint. Use the following input and output requirements for forming the architecture.

Define API endpoints also

Input:

Github PRs raised

JIRA Test Cases

JIRA User Stories

Confluence

Defect Database

Slack discussions

Github report

Technical documentation

Output:

Testcases

Impacted features

Ingestion Pipeline-injest all the existing testcases and userstories by creating embedding

Retrieval pipeline: Hybrid search, Rerank, Deduplication, Summarization

Prompting

LLM output

Verify the output

Vector database :MongoDB

embedding model:Mistral AI

Tech stack: Front end: [react.js](https://reactjs.org/), backend :Node and typescript

Feedback mechanism (Approve/reject/edit)

folder structure you can decide as per enterprise level and then design

Context :

You are an expertised mobile application tester and a test architect with 15+ years of expertise into car rental mobile app domain

Persona:

You act as an senior most QA professional to design a regression testing impacted areas and regression test cases to be incorporated in the regression suite in a car rental mobile application for a weekly release

Output format:

It should be in downloadable excel ,.csv,json.,etc format

Tone :

Formal and Professional.

Start with the MCQ questionnaire and ask me the relevant question to design the architecture document.

One question at a time and each question should have 3 options to choose.

If in case I don't know the answer among these 3 options ,I will let you know ,then you can choose best one

If I don't understand the context of the question ,I will ask you to explain, then after the explanation I will choose the answer.

[STRICT] Based on my answers only you are going to design the architecture file and give me in the .md formatDO NOT HALLUCINATE AND ASK ME THE PROPER QUESTIONS

Chat GPT response:

I will follow your process strictly:

- I will ask **one MCQ question at a time**.
- Each question will have **3 options (A/B/C)**.
- If you don't know the answer, tell me "**Choose for me**", and I will select the most suitable enterprise-grade option for a Car Rental Mobile App QA platform.
- If you don't understand a question, ask "**Explain**", and I will explain before you answer.
- I will **not design the architecture until all required questions are answered**.
- The final deliverable will be a detailed **Architecture Document (.md format)** including:
 - High-Level Architecture
 - API Design
 - Ingestion Pipeline
 - Retrieval Pipeline (Hybrid Search + Rerank + Dedup)
 - Prompt Engineering Layer
 - LLM Orchestration
 - Validation Framework
 - Feedback Mechanism
 - MongoDB Vector Database Design
 - Folder Structure (Enterprise Level)
 - React + NodeJS + TypeScript Architecture
 - Export APIs for CSV / JSON / Excel
 - Security & Governance
 - Sequence Diagrams
 - Data Models
 - Car Rental Domain-specific Regression Impact Analysis Flow

Question 1:

Question 1

How should the system identify impacted areas from newly merged GitHub PRs?

A. PR Description Only

- Analyze title, description, labels, linked Jira tickets.
- Fastest implementation.
- Lower accuracy.

B. Changed Files + Code Diff Analysis

- Analyze modified files, classes, APIs, components, services and code diffs.
- Map them to user stories and test cases.
- Medium complexity.
- High accuracy.

C. Full Repository Dependency Analysis

- Analyze changed files, code diff, dependency graph, API contracts, service relationships and downstream impacts.
- Highest accuracy.
- Highest implementation effort.

Which option do you choose?

A. PR Description Only

B. Changed Files + Code Diff Analysis

C. Full Repository Dependency Analysis

Answered B

Recorded.

✓ Answer Selected: B. Changed Files + Code Diff Analysis

This means the architecture will:

- Parse GitHub PRs and code diffs.
- Identify changed files, functions, APIs, screens, services, and modules.
- Map impacted code to Jira User Stories, existing test cases, defects, and documentation.
- Use this impact data to generate regression recommendations.

Question 2:

Question 2

What should be considered the primary source of truth for mapping features to regression test cases?

A. Jira Test Cases Only

- Uses existing QA assets.
- Simple implementation.
- May miss business context.

B. Jira User Stories + Jira Test Cases

- Links requirements and validations.
- Better feature traceability.
- Common enterprise approach.

C. Jira User Stories + Jira Test Cases + Confluence Documentation

- Combines requirements, test coverage, and business/technical documentation.
- Highest contextual understanding.
- Better LLM recommendations.

Which option do you choose?

A. Jira Test Cases Only

B. Jira User Stories + Jira Test Cases

C. Jira User Stories + Jira Test Cases + Confluence Documentation

Answered C

Recorded.

✅ **Answer Selected: C. Jira User Stories + Jira Test Cases + Confluence Documentation**

This will allow the architecture to:

- Build embeddings from requirements, acceptance criteria, test cases, and business/technical documentation.
- Improve impact analysis accuracy.
- Generate better regression recommendations with richer context.

Question 3:

How should the ingestion pipeline create embeddings in MongoDB Vector Search?

A. Separate Collections

- Create separate embeddings for:
 - Jira User Stories
 - Jira Test Cases
 - Confluence
 - Defects
 - GitHub Reports
 - Technical Documents
- Query each collection independently.

B. Unified Knowledge Collection

- Store all documents in a single vector collection.
- Use metadata (`source_type` , `feature` , `module` , etc.) for filtering.
- Simpler retrieval.

C. Hybrid Enterprise Model

- Dedicated collections per source.
- Plus a normalized knowledge graph/metadata layer.
- Supports source-specific retrieval and cross-source correlation.
- Most scalable enterprise architecture.

Which option do you choose?

A. Separate Collections

B. Unified Knowledge Collection

C. Hybrid Enterprise Model



Answered C

Recorded.

✓ **Answer Selected: C. Hybrid Enterprise Model**

This will enable:

- Separate collections for Jira Test Cases, User Stories, Confluence, Defects, GitHub Reports, and Technical Documents.
- A normalized metadata layer for feature/module mapping.
- Better traceability, scalability, and enterprise-grade retrieval.
- More accurate regression impact analysis across multiple knowledge sources.

Question 4:

Question 4

What retrieval strategy should be used to identify impacted regression test cases?

A. Pure Vector Search

- Semantic similarity only.
- Fast implementation.
- May miss exact keyword matches.

B. Hybrid Search (Vector + BM25)

- Combines semantic search and keyword search.
- Better recall.
- Industry standard for RAG systems.

C. Hybrid Search + Reranking

- Vector Search + BM25 Retrieval.
- Cross-Encoder Reranker.
- Returns most relevant impacted test cases and features.
- Highest accuracy.

Which option do you choose?

A. Pure Vector Search

B. Hybrid Search (Vector + BM25)

C. Hybrid Search + Reranking



Answered C

Recorded.

✓ Answer Selected: C. Hybrid Search + Reranking

The retrieval pipeline will therefore include:

1. Vector Search (Mistral Embeddings + MongoDB Vector Search)
2. BM25 Keyword Search
3. Hybrid Score Aggregation
4. Cross-Encoder Reranking
5. Deduplication
6. Context Summarization
7. LLM Recommendation Generation

This is typically the most accurate approach for regression impact analysis.

Question 5:

Question 5

How should the system generate new regression test cases when gaps are identified?

A. Generate Only from Impacted PR Context

- Uses PR diff and impacted modules.
- Faster.
- May miss existing business rules.

B. Generate from PR Context + Retrieved Knowledge

- Uses PR diff, user stories, existing test cases, defects, and documentation.
- Produces contextual regression recommendations.
- Recommended enterprise approach.

C. Generate from PR Context + Retrieved Knowledge + Historical Defect Patterns

- Uses everything in Option B.
- Also analyzes historical production defects, escaped defects, and defect trends.
- Highest likelihood of preventing regression leakage.

Which option do you choose?

A. PR Context Only

B. PR Context + Retrieved Knowledge

C. PR Context + Retrieved Knowledge + Historical Defect Patterns

Answered C

Recorded.

✓ **Answer Selected: C. PR Context + Retrieved Knowledge + Historical Defect Patterns**

This will allow the solution to:

- Analyze the PR code changes.
- Retrieve related Jira User Stories, Test Cases, Confluence pages, Technical Documentation, and Defects.
- Learn from historical defect leakage patterns.
- Recommend:
 - Impacted Features
 - Existing Regression Test Cases to Execute
 - New Regression Test Cases to Add
 - Risk Areas
 - Defect-Prone Areas

This is a strong enterprise-grade approach for weekly mobile app releases.

Question 6:

How should the LLM output be validated before showing recommendations to QA users?

A. No Validation Layer

- LLM output shown directly.
- Fastest implementation.
- Higher hallucination risk.

B. Rule-Based Validation

- Validate feature names, modules, Jira IDs, test case IDs, and document references.
- Reject unsupported recommendations.
- Moderate accuracy.

C. Multi-Layer Validation Framework

- Rule-Based Validation
- Confidence Scoring
- Source Attribution Verification
- Hallucination Detection
- Coverage Analysis
- QA Approval Workflow

This provides the strongest governance and auditability.

Which option do you choose?

A. No Validation Layer

B. Rule-Based Validation

C. Multi-Layer Validation Framework



Answered C

Recorded.

✓ Answer Selected: C. Multi-Layer Validation Framework

The architecture will include:

- Source Attribution Validation
- Confidence Scoring
- Hallucination Detection
- Requirement Coverage Validation
- Test Case Traceability Validation
- QA Approval Workflow
- Audit Trail & Explainability Layer

This is particularly important when recommendations may influence a production regression suite.

Question 7:

Question 7

How should user feedback (Approve / Reject / Edit) be utilized?

A. Store Feedback for Reporting Only

- Feedback visible in dashboards.
- No impact on future recommendations.

B. Feedback-Driven Prompt Optimization

- Approved and rejected recommendations influence prompt templates.
- Improves future generations.

C. Continuous Learning Loop

- Store feedback.
- Track edits and approvals.
- Build feature-wise confidence scores.
- Generate recommendation quality metrics.
- Use feedback during retrieval, reranking, and prompt construction.

This creates a self-improving regression intelligence platform.

Which option do you choose?

A. Reporting Only

B. Prompt Optimization

C. Continuous Learning Loop

Answered C

Recorded.

✓ Answer Selected: C. Continuous Learning Loop

The platform will become progressively smarter by:

- Capturing Approve / Reject / Edit actions.
- Tracking acceptance rates per feature/module.
- Building recommendation confidence scores.
- Improving retrieval and reranking quality.
- Optimizing prompt context assembly.
- Providing QA effectiveness metrics and audit trails.

Question 8:

What should be the primary output granularity for impacted feature identification?

A. Module Level

- Example:
 - Authentication
 - Booking
 - Payments
 - Profile

B. Feature Level

- Example:
 - Add Driver
 - Modify Reservation
 - Pay & Book
 - Cancel Booking
 - Vehicle Upgrade

C. Feature + User Journey Level

- Example:
 - Add Driver
 - Pay & Book
 - Modify Reservation
 - Plus impacted end-to-end journeys:
 - Guest Booking Journey
 - Logged-in Booking Journey
 - Driver Addition Journey
 - Vehicle Pickup Journey



Answered C

Recorded.

✅ **Answer Selected: C. Feature + User Journey Level**

This is especially valuable for a **car rental mobile application**, where many regressions occur not just within a feature, but across interconnected booking journeys.

The architecture will therefore identify:

- Impacted Modules
- Impacted Features
- Impacted APIs
- Impacted Mobile Screens
- Impacted User Journeys
- Impacted Regression Test Cases
- Recommended New Regression Coverage



Question 9:

Question 9

How should regression recommendations be exported for QA consumption?

A. CSV Only

- Simple implementation.
- Easy import into Jira/Excel.

B. Excel + CSV

- Most QA teams prefer Excel.
- Supports filters, tabs, and reporting.

C. Excel + CSV + JSON + REST API

- Excel for QA teams.
- CSV for bulk processing.
- JSON for integrations.
- REST APIs for CI/CD and external systems.

This is the most enterprise-ready option.

Which option do you choose?

A. CSV Only

B. Excel + CSV

C. Excel + CSV + JSON + REST API

Answered C

Recorded.

✅ **Answer Selected: C. Excel + CSV + JSON + REST API**

The final architecture will therefore support:

- **Excel (.xlsx)** for QA review and signoff.
- **CSV** for bulk analysis.
- **JSON** for integrations.
- **REST APIs** for CI/CD pipelines, dashboards, Jira integrations, and external tools.

Question 10:

What level of explainability should be provided for every impacted feature and recommended regression test case?

A. Basic

- Show impacted feature.
- Show recommended test cases.

B. Traceable

- Show impacted feature.
- Show recommended test cases.
- Show source Jira stories and PRs used.

C. Fully Explainable AI

- Show impacted feature.
- Show impacted user journeys.
- Show recommended existing regression cases.
- Show recommended new regression cases.
- Show source PRs.
- Show source Jira stories.
- Show source defects.
- Show retrieval score.
- Show rerank score.
- Show confidence score.
- Show reasoning generated by the validation layer 

This provides maximum trust for QA leads and release managers.

Answered C

Recorded.

✓ Answer Selected: C. Fully Explainable AI

The final architecture document will include:

- Impacted Features
- Impacted User Journeys
- Existing Regression Test Cases
- Recommended New Regression Test Cases
- Source GitHub PRs
- Source Jira User Stories
- Source Jira Test Cases
- Source Defects
- Source Confluence Pages
- Retrieval Scores
- Reranking Scores
- Confidence Scores
- Validation Results
- AI Reasoning & Explainability Layer
- QA Approval Audit Trail



Question 11:

Before I generate the final architecture document, I need one last implementation-critical question.

Question 11 (Final)

How should the system process merged GitHub PRs during a sprint?

A. On-Demand Processing

- QA clicks "Analyze Sprint".
- System processes all merged PRs and generates recommendations.

B. Event-Driven Processing

- Every merged PR triggers ingestion and impact analysis immediately.
- Results are accumulated throughout the sprint.

C. Hybrid Processing

- Every merged PR is analyzed immediately and stored.
- At sprint end (or on demand), a consolidated sprint-level regression recommendation is generated using all analyzed PRs.

This is generally the most scalable and release-friendly approach.

Answered A

Recorded.

✓ Answer Selected: A. On-Demand Processing

The architecture will therefore:

- Process all merged PRs for a selected sprint when a QA user initiates analysis.
- Perform batch ingestion of PR metadata and code diffs.
- Execute impact analysis across all sprint changes.
- Generate a consolidated regression recommendation package for the weekly release.
- Produce downloadable Excel/CSV/JSON outputs and API responses.

Architecture Design Readiness:

Architecture Design Readiness Check		
I now have enough information to generate the final architecture document. Below is the summary of your selections:		
Area	Selected Option	
PR Impact Analysis	Changed Files + Code Diff Analysis	
Source of Truth	Jira Stories + Jira Test Cases + Confluence	
Embedding Strategy	Hybrid Enterprise Model	
Retrieval	Hybrid Search + Reranking	
Test Generation	PR + Knowledge Base + Historical Defects	
Validation	Multi-Layer Validation Framework	
Feedback	Continuous Learning Loop	
Impact Identification	Feature + User Journey	
Export	Excel + CSV + JSON + REST APIs	
Explainability	Fully Explainable AI	
Processing Mode	On-Demand  Print Analysis	

Next Step
I am ready to generate the complete architecture document in Markdown (.md) format.
The document will include:
1. Executive Summary 2. High-Level Architecture Diagram (Mermaid) 3. Component Architecture 4. Ingestion Pipeline 5. MongoDB Vector Database Design 6. Mistral Embedding Strategy 7. Retrieval Pipeline 8. Hybrid Search Architecture 9. Reranking Architecture 10. Deduplication Framework

11. Summarization Layer
12. Prompt Engineering Framework
13. LLM Orchestration
14. Validation Framework
15. Feedback Learning Loop
16. API Endpoint Specifications
17. React Frontend Architecture
18. Node.js + TypeScript Backend Architecture
19. Enterprise Folder Structure
20. Database Collections
21. Output Formats (Excel/CSV/JSON)
22. Explainability Framework
23. Security & Audit Design
24. End-to-End Sequence Diagram
25. Car Rental Domain Example Flow
26. Sprint Regression Analysis Workflow

Please confirm with either:

- "Generate Architecture Document" → I will generate the complete `.md` document.
- "Add More Questions" → I will continue refining the design before generating it.